

Date:
Student Name:
Student ID:

AI Workshop #8

Heuristic Search Techniques

8.1. Introduction

In this workshop, we are going to learn about **Heuristic Search Techniques**.

Heuristic search techniques are used to search through the solution space to come up with answers. The search is conducted using heuristics that guide the search algorithm.

This heuristic allows the algorithm to speed up the process, which would otherwise take a long time to arrive at the solution.

By the end of this workshop, you will have a better understanding of these topics:

- What is heuristic search?
- Uninformed vs. informed search
- Constraint satisfaction problems
- Local search techniques
- Simulated annealing
- Constructing a string using greedy search
- Solving a problem with constraints
- Solving the region coloring problem
- Building an 8-puzzle solver
- Mini-project : Building a maze solver

8.2 Is Heuristic Search AI?

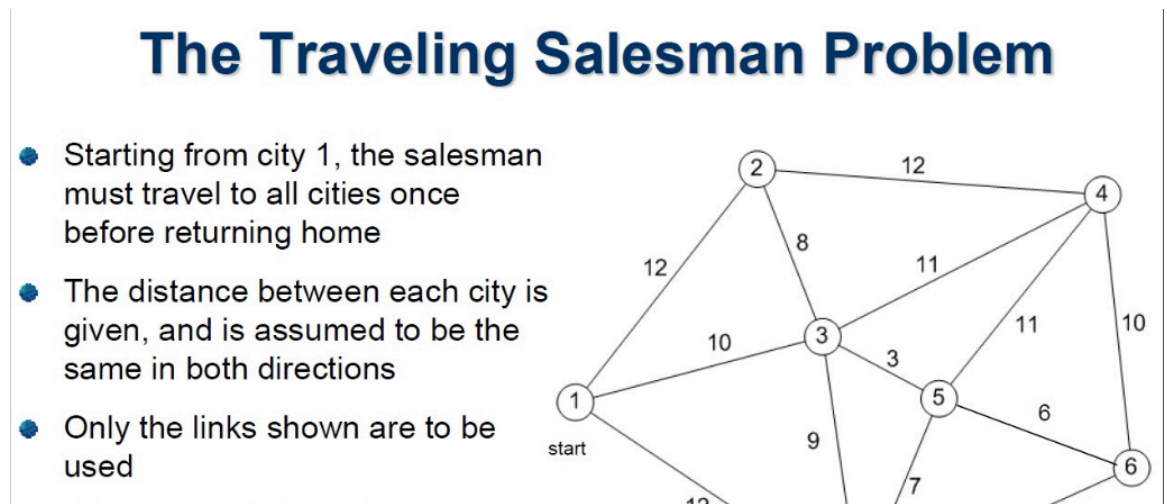
In semester Introduction to AI, we learnt basic concepts and knowledge on AI. One of the most "ancient" tribes is the symbolist tribe. At least to me, this fact is not surprising.

As humans, we try to find rules and patterns in everything. Unfortunately, the world is sometimes messy and not everything follows simple rules.

This is the reason why other tribes emerged to help us when we don't have an orderly world.

However, when our search spaces are small and the domain is limited, using heuristics, constraint satisfaction, and other techniques laid out in this workshop, is useful for this set of problems. These techniques are useful when the number of combinations is relatively small and combinatorial explosion is limited.

For example, solving the traveling salesman problem (TSP) is simple with these techniques when the number of cities is around 20. If we try to solve the same problem for $n=2000$, other techniques will have to be used that don't explore the complete space and only give an approximation of the result.



8.3 What is Heuristic Search?

Searching and organizing data is an important topic within artificial intelligence.

There are many problems that require searching for an answer within the solution domain. There are many possible solutions to a given problem and we do not know which ones are correct.

By efficiently organizing the data, we can search for solutions quickly and effectively.

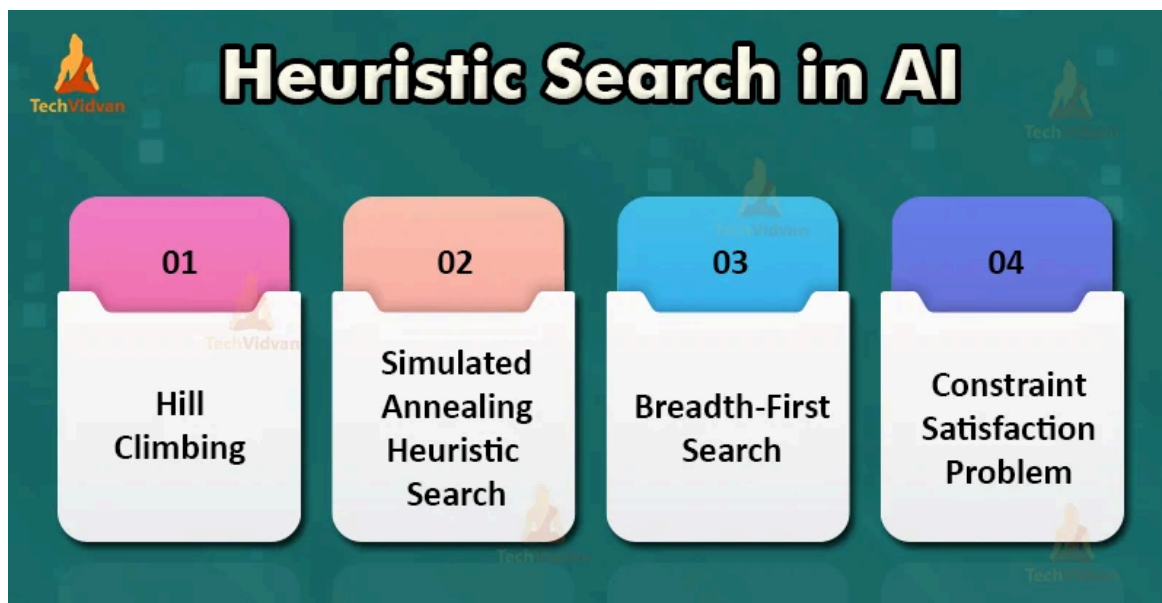
More often, there are so many possible options to solve a given problem that no single algorithm can be developed to find a definite best solution.

Also, going through every solution is not possible because it is prohibitively expensive. In such cases, we rely on a rule of thumb that helps us narrow down the search by eliminating the options that are obviously wrong.

This rule of thumb is called a **heuristic**. The method of using heuristics to guide the search is called **heuristic search**.

Heuristic techniques are powerful because they speed up the process. Even if the heuristic is not able to eliminate some options, it will help ordering those options so that better solutions are likely to come up first.

As mentioned previously, heuristic searches can be computationally expensive. We will now learn how we can take shortcuts and prune the **search tree**.



Uninformed versus Informed Search

If you are familiar with computer science, you might have heard about search techniques such as Depth First Search (DFS), Breadth First Search (BFS), and Uniform Cost Search (UCS). These are search techniques that are commonly used on graphs to get to the solution.

These are examples of uninformed searches. They do not use any prior information or rules to eliminate some paths. They check all the plausible paths and pick the optimal one.

A heuristic search, on the other hand, is called an **informed search** because it uses prior information or rules to eliminate unnecessary paths.

Uninformed search techniques do not take the goal into account. Uninformed search techniques search blindly and have no a priori knowledge about the final solution.

In the **graph problem**, heuristics can be used to guide the search.

For example, at each node we can define a heuristic function that returns a score that represents the estimate of the cost of the path from the current node to the goal. By defining this heuristic function, we are informing the search technique about the right direction to reach the goal. This will allow the algorithm to identify which neighbor will lead to the goal.

We need to note that heuristic searches might not always find the most **optimal solution**.

8.4 Constraint Satisfaction Problems

There are many problems that must be solved under constraints. Chess playing is a very example.

These constraints are basically conditions that cannot be violated during the process of solving the problem.

These problems are referred to as **Constraint Satisfaction Problems (CSPs)**.

To gain some intuitive understanding, let's quickly look at an example section of a **Sudoku puzzle**. Sudoku is a game where we cannot have the same number twice across a horizontal line, vertical line, or in the same square.

Here is an example Sudoku board:

			2	6		7		1
6	8			7			9	
1	9				4	5		
8	2		1				4	
		4	6		2	9		
	5				3		2	8
		9	3				7	4
	4			5			3	6
7		3		1	8			

Using constraint satisfaction and the rules of Sudoku we can quickly determine which numbers to try and which numbers not to try to solve the puzzle.

For example, in this square:

6	8		
1	9		

If we were not using CSP, one brute force approach would be to try all of the combinations of numbers in the slots and then check if the rules applied.

For example, our first attempt might be to fill out all the squares with the number 1, and then check the result.

Using CSP, we can prune the attempts before we try them.

Let's walk through what we think the number should be for the square highlighted in red. We know that the number cannot be a 1, 6, 8 or a 9 because those numbers already exist in the square. We also know that it cannot be a 2 or 7 because those number exist in the horizontal line. We also know that it cannot be a 3 or a 4 because those number are already in the vertical line. That leaves us with the only possibility of what the number should be – 5.

CSPs are mathematical problems that are defined as a set of variables that must satisfy some constraints. When we arrive at the final solution, the states of the variables must obey all the constraints. This technique represents the entities involved in a given problem as a collection of a fixed number of constraints over the variables. These variables need to be solved by constraint satisfaction methods.

These problems require a combination of heuristics and other search techniques to be solved in a reasonable amount of time. In this case, we will use constraint satisfaction techniques to solve problems on finite domains. A finite domain consists of a finite number of elements. Since we are dealing with finite domains, we can use search techniques to arrive at the solution. To get further clarity on CSP we will now learn how to use local search techniques to solve CSP problems.

8.5 Local Search Techniques

Local search is a way of solving a CSP.

It keeps optimizing the solution until all the constraints are satisfied. It iteratively keeps updating the variables until we arrive at the destination.

These algorithms modify the value during each step of the process that gets us closer to the goal. In the solution space, the updated value is closer to the goal than the previous value.

Hence, it is known as a **local search**.

A local search algorithm is a type of heuristic search algorithm.

These algorithms use a function that calculates the quality of each update.

For example, it can count the number of constraints that are being violated by the current update or it can see how the update affects the distance to the goal.

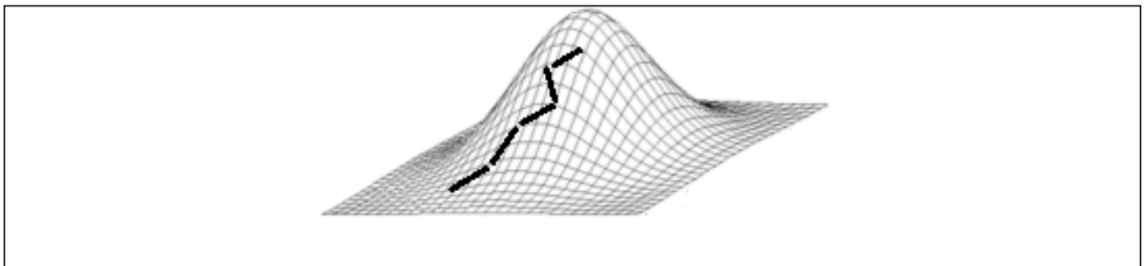
This is referred to as the **cost** of the assignment. The overall goal of local search is to find the **minimum cost** update at each step.

Hill Climbing Search Technique

Hill climbing is a popular local search technique.

It uses a heuristic function that measures the difference between the current state and the goal.

When we start, it checks if the state is the final goal. If it is, then it stops. If not, then it selects an update and generates a new state. If it's closer to the goal than the current state, then it makes that the current state. If not, it ignores it and continues the process until it checks all possible updates. It basically climbs the hill until it reaches the summit.



Simulated Annealing Search Technique

Simulated annealing is a type of local search, as well as a stochastic search technique. Stochastic search techniques are used extensively in various fields, such as robotics, chemistry, manufacturing, medicine, and economics. Stochastic algorithms are used to solve many real-world problems: we can perform things like optimizing the design of a robot, determining the timing strategies for automated control in factories, and planning traffic.

Simulated annealing is a variation of the hill climbing technique. One of the main problems of hill climbing is that it ends up climbing false foothills; this means that it gets stuck in local maxima. So, it is better to check out the whole space before we make any climbing decisions. In order to achieve this, the whole space is initially explored to see what it is like. This helps us avoid getting stuck in a plateau or local maxima.

In simulated annealing, we reformulate the problem and solve it for minimization, as opposed to maximization. So, we are now descending into valleys as opposed to climbing hills. We are pretty much doing the same thing, but in a different way.

We use an **objective function** to guide the search. This objective function serves as the heuristic.



The reason it is called simulated annealing is because it is derived from the metallurgical process. In that process, we first heat metals up, allowing atoms to diffuse within the metal, and then let them cool until they reach their optimal desired state in terms of atomic structural arrangement. This is typically to change a metal's physical properties to become softer and easier to work.

The rate at which we cool the system is called the annealing schedule. The rate of cooling is important because it directly impacts the result. In the real-world case of metals, if the rate of cooling is too fast, it ends up settling too quickly into a nonideal state (atomic structure). For example, if the heated metal is put into cold water, it ends up quickly settling into a structure that was not desired, making the metal brittle, for instance.

If the rate of cooling is slow and controlled, the metal has a chance to arrive at the optimal atomic structure, leading to the desired physical properties. The chances of taking big steps quickly towards any hill are lower in this case. Since the rate of cooling is slow, it will take its time to move into the optimal state. Something similar can be done with data.

We first evaluate the current state and see if it has reached the goal. If it has, then we stop. If not, then we set the best state variable to the current state. We then define an annealing schedule that controls how quickly it descends into a valley. The difference is computed between the current state and the new state. If the new state is not better, then we make it the current state with a certain predefined probability.

This is done using a random number generator and deciding based on a threshold. If it is above the threshold, then we set the best state to this state. Based on this, the annealing schedule is updated depending on the number of nodes. We keep doing this until we arrive at the goal. Another local search technique is the greedy search algorithm. We will learn more about that in the next section.

8.6 Constructing a String using Greedy Search

Greedy search is an algorithmic paradigm that makes the locally optimal choice at each stage in order to find the global optimum. But in many problems, greedy algorithms do not produce globally optimum solutions. An advantage of using greedy algorithms is that they produce an approximate solution in a reasonable time. The hope is that this approximate solution is reasonably close to the global optimal solution.

Greedy algorithms do not refine their solutions based on new information during the search. For example, let's say you are planning on a road trip and you want to take the best route possible. If you use a greedy algorithm to plan the route, it might ask you to take routes that have a shorter distance but might end up taking more time. It may also lead you to paths that may seem faster in the short term but might lead to traffic jams later. This happens because greedy algorithms only see the next step and not the globally-optimal final solution.

Let's see how to solve a problem using a greedy search. In this problem, we will try to recreate the input string based on the alphabets. We will ask the algorithm to search the solution space and construct a path to the solution.

We will be using a package called `simpleai` throughout this workshop. It contains various routines that are useful in building solutions using heuristic search techniques. It's available at <https://github.com/simpleai-team/simpleai> (<https://github.com/simpleai-team/simpleai>). We need to make a few changes to the source code in order to make it work in Python 3.

A file called **simpleai.zip** has been provided. Unzip this file into a folder called simpleai. This folder contains all the necessary changes to the original library necessary to make it work in Python 3. Place the **simpleai** folder in the same folder as your code and you'll be able to run

1. Create a new Python file and import the following packages:

```
import argparse
import simpleai.search as ss
```

2. Define a function to parse the input arguments:

```
def build_arg_parser():
    parser = argparse.ArgumentParser(description='Creates the input string \
        using the greedy algorithm')
    parser.add_argument("--input-string", dest="input_string",
        required=True,
        help="Input string")
    parser.add_argument("--initial-state", dest="initial_state",
        required=False,
        default='', help="Starting point for the search")
    return parser
```

3. Create a class that contains the methods needed to solve the problem. This class inherits the SearchProblem class available in the library. A few methods need to be overridden to solve the problem at hand. The first method set_target is a custom method that defines the target string:

```
class CustomProblem(ss.SearchProblem):
    def set_target(self, target_string):
        self.target_string = target_string
```

4. The actions is a method that comes with a SearchProblem and that needs to be overridden. It's responsible for taking the right steps towards the goal. If the length of the current string is less than the length of the target string, it will return the list of possible alphabets to choose from. If not, it will return an empty string:

```
# Check the current state and take the right action
def actions(self, cur_state):
    if len(cur_state) < len(self.target_string):
        alphabets = 'abcdefghijklmnopqrstuvwxyz'
        return list(alphabets + ' ' + alphabets.upper())
    else:
        return []
```

5. Now create a method to compute the result by concatenating the current string and the action that needs to be taken. This method comes with a SearchProblem and we are overriding it:

```
# Concatenate state and action to get the result
def result(self, cur_state, action):
```

```
return cur_state + action
```

6. The method `is_goal` is a part of the `SearchProblem` and it's used to check if the goal has been reached:

```
# Check if goal has been achieved
def is_goal(self, cur_state):
    return cur_state == self.target_string
```

7. The method `heuristic` is also a part of the `SearchProblem` and we need to override it. A heuristic is defined that will be used to solve the problem. A calculation is performed to see how far the goal is and use that as the heuristic to guide it towards the goal:

```
# Define the heuristic that will be used
def heuristic(self, cur_state):
    # Compare current string with target string
    dist = sum([1 if cur_state[i] != self.target_string[i] else 0
                for i in range(len(cur_state))])

    # Difference between the lengths
    diff = len(self.target_string) - len(cur_state)

    return dist + diff
```

8. Initialize the input arguments:

```
if __name__ == '__main__':
    args = build_arg_parser().parse_args()
```

9. Initialize the `CustomProblem` object:

```
# Initialize the object
problem = CustomProblem()
```

10. Set the starting point as well as the goal we want to achieve:

```
# Set target string and initial state
problem.set_target(args.input_string)
problem.initial_state = args.initial_state
```

11. Run the solver:

```
# Solve the problem
output = ss.greedy(problem)
```

12. Print the path to the solution:

```
print('\nTarget string:', args.input_string)
print('\nPath to the solution:')

```

```
for item in output.path():
    print(item)
```

13. If you run the code with an empty initial state, like this:

```
python greedy_search.py --input-string "Artificial Intelligence" --initial-state ""
```

You will get this output:

```
Target string: Artificial Intelligence
Path to the solution:
(None, '')
('A', 'A')
('r', 'Ar')
('t', 'Art')
('i', 'Arti')
('f', 'Artif')
('i', 'Artifi')
('c', 'Artific')
('i', 'Artifici')
('a', 'Artificia')
('l', 'Artificial')
(' ', 'Artificial ')
('I', 'Artificial I')
('n', 'Artificial In')
('t', 'Artificial Int')
('e', 'Artificial Inte')
('l', 'Artificial Intel')
('l', 'Artificial Intell')
('i', 'Artificial Intelli')
('g', 'Artificial Intellig')
('e', 'Artificial Intellige')
('n', 'Artificial Intelligen')
('c', 'Artificial Intelligenc')
('e', 'Artificial Intelligence')
```



Workshop 8.1 : Greedy Search for String Construction

1. Combine all the above code and named it as **greedy_search.py**.
2. Try the first test as above using empty initial state, see whether you got the same output:

```
python greedy_search.py --input-string "Artificial Intelligence" --initial-state ""
```

3. In the second test, run the code with a non-empty starting point:

```
python greedy_search.py --input-string "Artificial Intelligence with Python" --initial-state "Artificial Inte"
```

4. What is your output?
5. Save and submit the code and these two outputs.
6. Use this method, explain how the Greedy Method works for String construction.

8.7 Solving a problem with constraints

We have already discussed how CSPs are formulated.

Let's apply them to a realworld problem. In this problem, we have a list of names and each name can take a fixed set of values.

We also have a set of constraints between these people that needs to be satisfied. Let's see how to do it.

1. Create a new Python file and import the following packages:

```
from simpleai.search import CspProblem, backtrack, \
    min_conflicts, MOST_CONSTRAINED_VARIABLE, \
    HIGHEST_DEGREE_VARIABLE, LEAST_CONSTRAINING_VALUE
```

2. Define the constraint that specifies that all the variables in the input list should have unique values:

```
# Constraint that expects all the different variables
# to have different values
def constraint_unique(variables, values):
    # Check if all the values are unique
    return len(values) == len(set(values))
```

3. Define the constraint that specifies that the first variable should be bigger than the second variable:

```
# Constraint that specifies that one variable
# should be bigger than other
def constraint_bigger(variables, values):
    return values[0] > values[1]
```

4. Define the constraint that specifies that if the first variable is odd, then the second variable should be even and vice versa:

```
# Constraint that specifies that there should be
# one odd and one even variables in the two variables
def constraint_odd_even(variables, values):
    # If first variable is even, then second should
    # be odd and vice versa
    if values[0] % 2 == 0:
```

```

        return values[1] % 2 == 1
    else:
        return values[1] % 2 == 0

```

5. Define the main function and define the variables:

```

if __name__ == '__main__':
    variables = ('John', 'Anna', 'Tom', 'Patricia')

```

6. Define the list of values that each variable can take:

```

domains = {
    'John': [1, 2, 3],
    'Anna': [1, 3],
    'Tom': [2, 4],
    'Patricia': [2, 3, 4],
}

```

7. Define the constraints for various scenarios. In this case, we specify three constraints as follows:

- John, Anna, and Tom should have different values
- Tom's value should be bigger than Anna's value
- If John's value is odd, then Patricia's value should be even and vice versa

```

constraints = [
    (('John', 'Anna', 'Tom'), constraint_unique),
    (('Tom', 'Anna'), constraint_bigger),
    (('John', 'Patricia'), constraint_odd_even),
]

```

8. Use the preceding variables and the constraints to initialize the CspProblem object:

```

problem = CspProblem(variables, domains, constraints)

```

9. Compute the solution and print it:

```

print('\nSolutions:\n\nNormal:', backtrack(problem))

```

10. Compute the solution using the MOST_CONSTRAINED_VARIABLE heuristic:

```

print('\nSolutions:\n\nNormal:', backtrack(problem))
print('\nMost constrained variable:', backtrack(problem,
    variable_heuristic=MOST_CONSTRAINED_VARIABLE))

```

11. Compute the solution using the HIGHEST_DEGREE_VARIABLE heuristic:

```

print('\nHighest degree variable:', backtrack(problem,

```

```
variable_heuristic=HIGHEST_DEGREE_VARIABLE))
```

12. Compute the solution using the LEAST_CONSTRAINING_VALUE heuristic:

```
print('\nLeast constraining value:', backtrack(problem,  
value_heuristic=LEAST_CONSTRAINING_VALUE))
```

13. Compute the solution using the MOST_CONSTRAINED_VARIABLE variable heuristic and the LEAST_CONSTRAINING_VALUE value heuristic:

```
print('\nMost constrained variable and least constraining value:',  
backtrack(problem,  
variable_heuristic=MOST_CONSTRAINED_VARIABLE,  
value_heuristic=LEAST_CONSTRAINING_VALUE))
```

14. Compute the solution using the HIGHEST_DEGREE_VARIABLE variable heuristic and the LEAST_CONSTRAINING_VALUE value heuristic:

```
print('\nHighest degree and least constraining value:',  
backtrack(problem, variable_heuristic=HIGHEST_DEGREE_VARIABLE,  
value_heuristic=LEAST_CONSTRAINING_VALUE))
```

15. Compute the solution using the minimum conflicts heuristic:

```
print('\nMinimum conflicts:', min_conflicts(problem))
```

16. If everything goes fine, you will see these outputs:

Solutions:

Normal: {'John': 1, 'Anna': 3, 'Tom': 4, 'Patricia': 2}

Most constrained variable: {'Anna': 1, 'Tom': 2, 'John': 3, 'Patricia': 2}

Highest degree variable: {'John': 1, 'Anna': 3, 'Tom': 4, 'Patricia': 2}

Least constraining value: {'John': 1, 'Anna': 3, 'Tom': 4, 'Patricia': 2}

Most constrained variable and least constraining value: {'Anna': 1, 'Tom': 2, 'John': 3, 'Patricia': 2}

Highest degree and least constraining value: {'John': 1, 'Anna': 3, 'Tom': 4, 'Patricia': 2}

Minimum conflicts: {'John': 3, 'Anna': 1, 'Tom': 4, 'Patricia': 4}

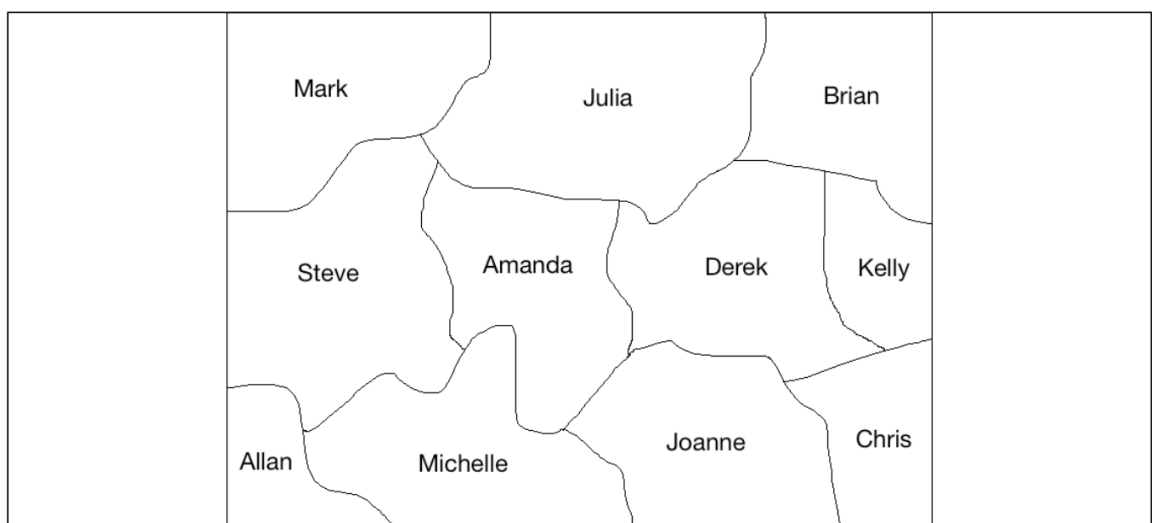


Workshop 8.2 : Problem Solving with CSP

1. Combine all the above code and named it as **csp_solver.py**.
2. Try to run the program, see whether you get the same results.
3. Save and submit the program code and results.
4. Invent TWO more constraints and name the program as **csp_solver_plus.py** (Note: You can change the sample data to increase more numbers for the new constraints to work)
5. Run it and check whether it works.
6. Save and submit this revised program and output results.

8.8 Solving the Region-Coloring Problem

Let's use the constraint satisfaction framework to solve the region-coloring problem. Consider the following figure:



We have a few regions in the preceding figure that are labeled with names. The goal is to color with four colors so that no adjacent regions have the same color.

1. Create a new Python file and import the following packages:

```
from simpleai.search import CspProblem, backtrack
```

2. Define the constraint that specifies that the values should be different:

```
# Define the function that imposes the constraint
# that neighbors should be different
def constraint_func(names, values):
    return values[0] != values[1]
```

3. Define the main function and specify the list of names:

```
if __name__ == '__main__':
    # Specify the variables
    names = ('Mark', 'Julia', 'Steve', 'Amanda', 'Brian',
```

```
'Joanne', 'Derek', 'Allan', 'Michelle', 'Kelly')
```

4. Define the list of possible colors:

```
# Define the possible colors
colors = dict((name, ['red', 'green', 'blue', 'gray']) for name in
names)
```

5. We need to convert the map information into something that the algorithm can understand. Let's define the constraints by specifying the list of people who are adjacent to each other:

```
# Define the constraints
constraints = [
    (('Mark', 'Julia'), constraint_func),
    (('Mark', 'Steve'), constraint_func),
    (('Julia', 'Steve'), constraint_func),
    (('Julia', 'Amanda'), constraint_func),
    (('Julia', 'Derek'), constraint_func),
    (('Julia', 'Brian'), constraint_func),
    (('Steve', 'Amanda'), constraint_func),
    (('Steve', 'Allan'), constraint_func),
    (('Steve', 'Michelle'), constraint_func),
    (('Amanda', 'Michelle'), constraint_func),
    (('Amanda', 'Joanne'), constraint_func),
    (('Amanda', 'Derek'), constraint_func),
    (('Brian', 'Derek'), constraint_func),
    (('Brian', 'Kelly'), constraint_func),
    (('Joanne', 'Michelle'), constraint_func),
    (('Joanne', 'Amanda'), constraint_func),
    (('Joanne', 'Derek'), constraint_func),
    (('Joanne', 'Kelly'), constraint_func),
    (('Derek', 'Kelly'), constraint_func),
]
```

6. Use the variables and constraints to initialize the object:

```
# Solve the problem
problem = CspProblem(names, colors, constraints)
```

7. Solve the problem and print the solution:

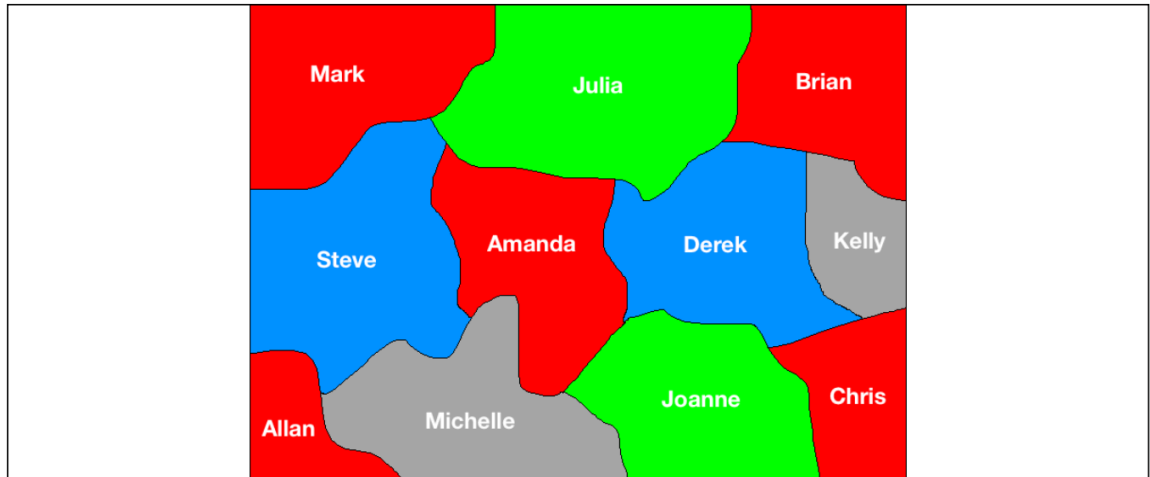
```
# Print the solution
output = backtrack(problem)
print('\nColor mapping:\n')
for k, v in output.items():
    print(k, '==>', v)
```

8. If you run the program, you should get it results:

Color mapping:

```
Mark ==> red
Julia ==> green
Steve ==> blue
Amanda ==> red
Brian ==> red
Joanne ==> green
Derek ==> blue
```

9. If you really coloring them, you will see NO two adjacent regions have the same color.



Workshop 8.3 : Region-Coloring Problem

1. Combine all the above code and named it as **coloring.py**.
2. Try to run the program, see whether you get the same results.
3. Save and submit the program code and results.
4. Does 4 coloring is the optimal solution? To test for it, try to modify the problem to test from 2 - 4 which one is the optimal solution, save and submit this problem as **coloring_test.py**.
5. In fact, region-coloring problem is a well known problem in mathematics and Graph Theory, especially on the coloring of map. Use the following map as example, design a new program **country_coloring.py** to color this map with the least number of color.



8.9 Building an 8-puzzle solver

The 8-puzzle is a variant of the 15-puzzle. You can check it out at https://en.wikipedia.org/wiki/15_puzzle (https://en.wikipedia.org/wiki/15_puzzle). You will be presented with a randomized grid and your goal is to get it back to the original ordered configuration. You can play the game to get familiar with it at <http://mypuzzle.org/sliding> (<http://mypuzzle.org/sliding>).

We will use an *A** algorithm to solve this problem. It is an algorithm that's used to find paths to the solution in a graph. This algorithm is a combination of Dijkstra's algorithm and a greedy best-first search. Instead of blindly guessing where to go next, the A* algorithm picks the one that looks the most promising. At each node, the list of all possibilities is generated and then the one with the minimal cost required to reach the goal is picked.

Let's see how to define the cost function. At each node, the cost needs to be computed. This cost is basically the sum of two costs – the first cost is the cost of getting to the current node and the second cost is the cost of reaching the goal from the current node.

We use this summation as a heuristic. As we can see, the second cost is basically an estimate that's not perfect. If it was perfect, then the A* algorithm would arrive at the solution quickly. But it's not usually the case. It takes some time to find the best path to the solution. A* is effective in finding the optimal paths, however, and is one of the most popular techniques out there.

Let's use the A* algorithm to build an 8-puzzle solver. This is a variant of the solution given in the **simpleai** library.

1. Create a new Python file and import the following packages:

```
from simpleai.search import astar, SearchProblem
```

2. Define a class that contains the methods to solve the 8-puzzle:

```
# Class containing methods to solve the puzzle
```

```
class PuzzleSolver(SearchProblem)
```

3. Override the actions method to align it with the current problem:

```
# Action method to get the list of the possible
# numbers that can be moved in to the empty space
def actions(self, cur_state):
    rows = string_to_list(cur_state)
    row_empty, col_empty = get_location(rows, 'e')
```

4. Check the location of the empty space and create the new action:

```
actions = []
if row_empty > 0:
    actions.append(rows[row_empty - 1][col_empty])
if row_empty < 2:
    actions.append(rows[row_empty + 1][col_empty])
if col_empty > 0:
    actions.append(rows[row_empty][col_empty - 1])
if col_empty < 2:
    actions.append(rows[row_empty][col_empty + 1])

return actions
```

5. Override the result method. Convert the string to a list and extract the location of the empty space. Generate the result by updating the locations:

```
# Return the resulting state after moving a piece to the empty space
def result(self, state, action):
    rows = string_to_list(state)
    row_empty, col_empty = get_location(rows, 'e')
    row_new, col_new = get_location(rows, action)

    rows[row_empty][col_empty], rows[row_new][col_new] = \
        rows[row_new][col_new], rows[row_empty][col_empty]

    return list_to_string(rows)
```

6. Check if the goal has been reached:

```
# Returns true if a state is the goal state
def is_goal(self, state):
    return state == GOAL
```

7. Define the heuristic method. We will use the heuristic that computes the distance between the current state and goal state using Manhattan distance:

```
# Returns an estimate of the distance from a state to
# the goal using the manhattan distance
def heuristic(self, state):
    rows = string_to_list(state)
```

```

        distance = 0

        for number in '12345678e':
            row_new, col_new = get_location(rows, number)
            row_new_goal, col_new_goal = goal_positions[number]

            distance += abs(row_new - row_new_goal) + abs(col_new -
col_new_goal)

        return distance

```

8. Define a function to convert a list to string:

```

# Convert list to string
def list_to_string(input_list):
    return '\n'.join(['-'.join(x) for x in input_list])

```

9. Define a function to convert a string to a list:

```

# Convert string to list
def string_to_list(input_string):
    return [x.split('-') for x in input_string.split('\n')]

```

10. Define a function to get the location of a given element in the grid:

```

# Find the 2D location of the input element
def get_location(rows, input_element):
    for i, row in enumerate(rows):
        for j, item in enumerate(row):
            if item == input_element:
                return i, j

```

11. Define the initial state and the final goal we want to achieve:

```

# Final result that we want to achieve
GOAL = '''1-2-3
4-5-6
7-8-e'''

# Starting point
INITIAL = '''1-e-2
6-3-4
7-5-8'''

```

12. Track the goal positions for each piece by creating a variable:

```

# Create a cache for the goal position of each piece
goal_positions = {}
rows_goal = string_to_list(GOAL)
for number in '12345678e':

```

```
goal_positions[number] = get_location(rows_goal, number)
```

13. Create the A* solver object using the initial state we defined earlier and extract the result:

```
# Create the solver object
result = astar(PuzzleSolver(INITIAL))
```

14. Print the solution:

```
# Print the results
for i, (action, state) in enumerate(result.path()):
    print()
    if action == None:
        print('Initial configuration')
    elif i == len(result.path()) - 1:
        print('After moving', action, 'into the empty space. Goal
achieved!')
    else:
        print('After moving', action, 'into the empty space')

    print(state)
```

15. If you run the program, you will see these running results:-

```
Initial configuration
1-e-2
6-3-4
7-5-8

After moving 2 into the empty space
1-2-e
6-3-4
7-5-8

After moving 4 into the empty space
1-2-4
6-3-e
7-5-8

After moving 3 into the empty space
1-2-4
6-e-3
7-5-8

After moving 6 into the empty space
1-2-4
e-6-3
7-5-8

After moving 1 into the empty space
e-2-4
1-6-3
7-5-8

After moving 2 into the empty space
2-e-4
1-6-3
7-5-8
```

16. If you scroll down, you will see the steps taken to arrive at the solution. At the end, you will see the following:

```
After moving 4 into the empty space
2-e-3
1-4-6
7-5-8
```

```
After moving 2 into the empty space
e-2-3
1-4-6
7-5-8
```

```
After moving 1 into the empty space
1-2-3
e-4-6
7-5-8
```

```
After moving 4 into the empty space
1-2-3
4-e-6
7-5-8
```

```
After moving 5 into the empty space
1-2-3
4-5-6
7-e-8
```

```
After moving 8 into the empty space. Goal achieved!
1-2-3
4-5-6
7-8-e
```



Workshop 8.4 : 8-Puzzle Solver

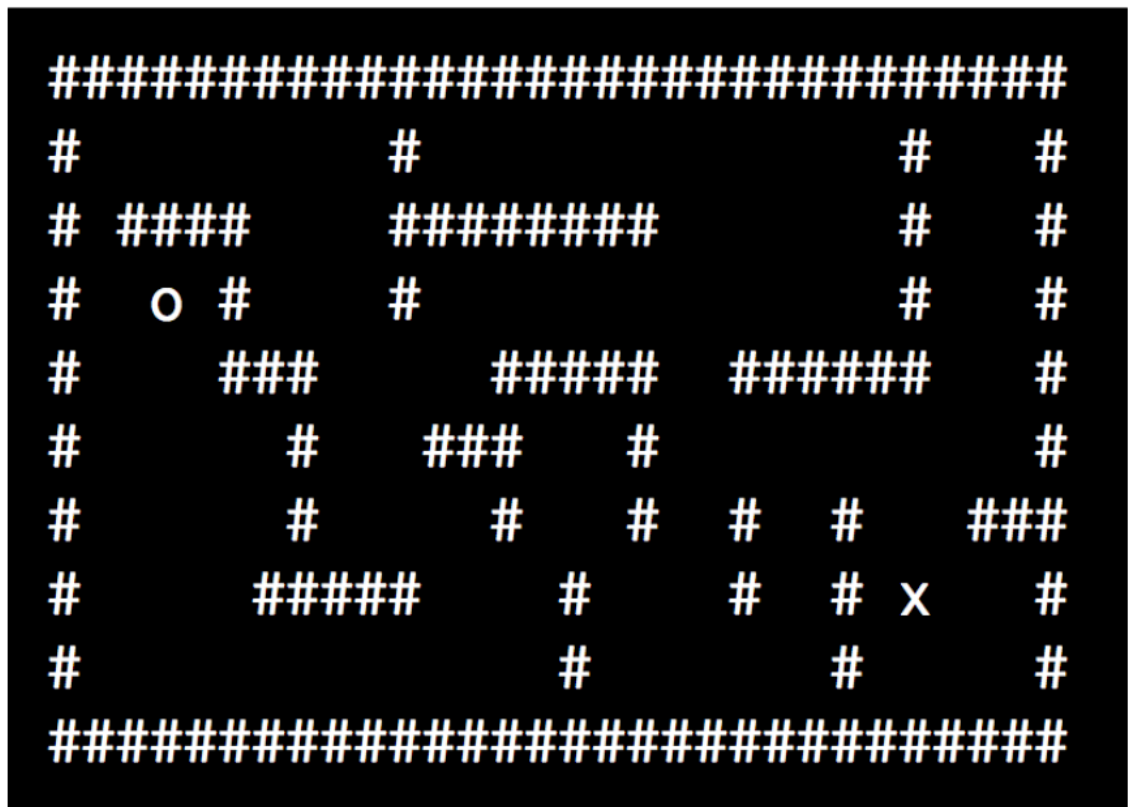
1. Combine all the above code and named it as **puzzle_solver.py**.
2. Try to run the program, see whether you get the same results.
3. Save and submit the program code and results.
4. Spend some time to study and learn how it works to prepare for the coming mini-project.



8.10 Mini-project: Building a Maze Solver

So far, we have learnt all the basic about Heuristic Search.

Let's use the A* algorithm to solve a maze. Consider the following figure:



The # symbols indicate obstacles. The symbol o represents the starting point, and x represents the goal. The goal is to find the shortest path from the start to the end point.

The solution should be something like this:

